

users consuming services. Therefore, we cannot afford to incur downtime to configure the replication.

Non-Goals

- The decision of which organization lives in which cell.
- Support for self-hosted installations.
- Geo may or may not be configured, or partially configured, on GitLab.com. This document does not require a decision at the time of writing.

High-Level Proposal

A organization move can be broken down into five distinct high-level phases:

1. Continuously replicate an organization's non-PostgreSQL data from the source cell to the target cell.

1. Org Mover will reuse Geo code to transfer non-PostgreSQL data continuously to the target cell.

The purpose is to reuse GitLab Geo code to replicate organization data outside the PostgreSQL database (files, Git data, object storage, container registry, etc.) from the source cell to the target cell. Given that cells cannot use PostgreSQL streaming replication, we need to make Geo independent of the method of PostgreSQL replication. This way, the target cell can connect directly to the source cell database or use PostgreSQL logical replication.

1. Prepare the organization for the switchover.

1. Put the organization into a maintenance mode.

A switchover requires a maintenance window in which updates to the source cell are blocked to ensure the switchover can occur without data loss. To block new writes to the data on the source Cell, we put the organization in maintenance mode. This means that the organization's users cannot make any writes to it while in maintenance mode.

The maintenance window ends once the replication of all the organization's data is completely finished and verified and the routing table is updated. So, to keep the window as short as possible, we should ensure that non-PostgreSQL data replication and verification processes are as close to 100% as possible during active use, and that the newer changes are almost immediately replicated to the target cell before putting the organization in maintenance mode.

1. Copy organization's PostgreSQL data from the source cell to the target cell.

1. Copy data for a particular organization from the source database and write it to the target database present in another cell.

Copy data for a particular organization from the source database in multiple tables based on organizationid and write it to the database present in the target cell.

Early Org Mover will implement a tool that uses the COPY approach to transfer PostgreSQL data during cutover. This requires a long downtime to move large organizations, but can be very quick for small organizations.

The tool is also responsible for the secrets/tokens stored at rest (database). It will establish a trust between the source and target cell to either re-encrypt data inline or prepare the encrypted data on the source cell to be re-encrypted with a transport key and re-encrypted to the target cell after the process finishes. This document does not require a decision at the time of writing.

1. Handle tables exempt from sharding.

Some tables are missing the sharding key, so the Org Mover will need to know how to move the data in each of these tables to the target database. This must happen after the organization database data copy is complete.

1. Handle the ElasticSearch and Zoekt data.

TBD

1. Rebuild the ClickHouse data.

We don't have the organizationid available in ClickHouse to move the data to another cell, and most data in ClickHouse is built from PostgreSQL. So once the organization database data copy is complete, we must rebuild the ClickHouse data from PostgreSQL data.

It is important to note that the Analytics Data Working Group is investigating different ways of syncing data, which would automatically pick up database-level changes and apply them to ClickHouse databases. Since the Org Mover re-creates data on the target cell and drops data from the source cell, changes will be picked up by ClickHouse automatically, and the Org Mover does not need to know about ClickHouse. This is still in the research phase.

1. Switchover and update the routing information.

There are no new writes to the organization data in the source cell, and the organization database in the source cell is equal to the organization data in the target cell. So, we change the routing information for the organization cellid. When we update the cellid, it will automatically make the given cell authoritative to handle traffic for the given organization, and we can disable the maintenance mode.

1. Remove all organization data that has been moved over.

1. Delete organization's stale data from the source cell:

1. Remove all data from the PostgreSQL database.

1. Remove all data from tables exempt from sharding.

1. Rebuild the ClickHouse data.

1. Remove all non-PostgreSQL data.

1. Delete organization's stale data from the target cell:

1. Remove all data from the Geo tracking database.

Implementation Roadmap

Based on roadmap planning as of April 2025, the Org Mover implementation will proceed through several phases:

FY26 Q1-Q2: Design and Foundational Work

1. Org Mover Design (FY25 Q4 - FY26 Q2)
 - Run POCs to validate technical approach
 - Investigate, discuss, and make critical design decisions
 - Define the interfaces and components
2. Org Mover Foundational Work (FY26 Q1 - FY26 Q2)
 - Refactor, fix, and improve scalability of Geo code and data structures for Org Mover on .com
 - Prepare the existing codebase for the requirements of organization-level operations
3. Org Mover Org Checksums 1 (FY26 Q2-Q3)
 - Build selective checksumming by organization
 - Ignore tables where sharding keys are not available

FY26 Q3-Q4: Implementation and Testing

1. Org Mover Partial PG Copy (FY26 Q3)
 - Implement functionality to move an organization's PostgreSQL data from one cell to another in GDK
 - Ignore tables where sharding keys are not available
2. Org Mover Org Checksums 2 (FY26 Q4)
 - Finish building selective checksumming by organization
 - Depends on completion of sharding work
3. Org Mover Org Checksums 3 (FY26 Q4)
 - Selectively roll out checksumming per component on GitLab.com
 - Validate checksumming features on .com
 - Let SREs begin cleaning up data on .com

FY27 Q1-Q2: Production Validation

1. Org Mover Test Org Move (FY27 Q1)
 - Move test organizations on GitLab.com
 - Validate end-to-end process in production
 - Identify and address any issues discovered in production
 - Repeat as frequently as possible

2. Org Mover 1: Internal Org Move (FY27 Q1)
 - Move an internal organization on GitLab.com that we actually use
 - Depends on a way to block user writes to the organization
 - Depends on organization data deletion, so we can reset the destination after each dry run
 - Validate production performance with our own real data
3. Org Mover 2: Customer Org Move (FY27 Q2)
 - Move a real customer organization on GitLab.com

FY27 Q2-FY27 Q4: Downsize Legacy Cell

1. Org Mover 3: Minimize Toil (FY27 Q2-Q3)
 - Automate Org Mover operations to allow for multiple concurrent moves with minimal manual labor
2. Org Mover 4: Reduce downtime with PG Logical Replication (FY27 Q4)
 - Implement PostgreSQL logical replication for organization moves
 - Reduce downtime during cutover phase
 - Note: May increase load on legacy database, so we expect logical replication to not be usable until many organizations have already been moved

FY28: Cell Rebalancing

1. Org Mover From Any Cell (FY28 Q1)
 - Support manual bi-directional moves (not just from legacy cell)

Organization Isolation and Sharding Dependencies

The Org Mover project has key dependencies on other Tenant Scale initiatives:

1. Sharding Work Completion (FY26 Q1-Q3)
 - Required for clean, performant data separation during migration
 - Required for Org Mover Org Checksums 2
2. Organization Data Isolation (FY26 Q4)
 - Required for clean, performant data separation during migration
3. Org on new cell (FY26 Q4)
 - Required for testing an organization that was migrated to a new cell

Migration approaches at different major iterations of Cells

As we build out Cells, we need a way to test the functionality at each major iteration. We want to dog food the new Cells deployment together with Organizations. The first iteration of Cells will have limited functionality, therefore, we cannot move all GitLab top-level groups out of the Legacy Cell to another Cell without severely impacting productivity. The following section outlines our approach to moving select top-level groups off the Legacy Cell into another Cell for dog food purposes, and how we reconcile the top-level group as the Cells development progresses through the iterations.

We will need to adopt different approaches for each major iteration of the Cells evolution based on which technologies are available at each iteration.

As we discuss creating and migrating Organizations, it's important to call out that top-level groups will retain their URL as they move between Organizations. For example, when a customer's top-level group is moved from the default Organization to their own Organization, their method of access remains unchanged. This means bookmarks, Git remote URLs, etc., for projects in these top-level groups remain unchanged and require no action from users or updates to automation after a migration.

Cells 1.0

Cells 1.0 will use Direct Transfer (DT) to move internal top-level groups to a separate Organization.

We still need to evaluate and address any gaps, but this is the only option available in this time frame.

DT is a copy instead of a move operation.

This means that DT will generate new IDs when the data is imported into a new Organization. It does not make sense to evolve DT to work as a move operation, because its core purpose is to import and export data.

It makes sense for DT to evolve to work with Organizations in time for Cells 1.0. This is not a throwaway effort, because DT will need to be supported when we have Cells and Organizations.

Limitations:

- Only top-level groups can be moved between organizations.
- New IDs will be generated which will break automation and integrations - This is acceptable since we will only be moving a limited set of internal top-level groups.
- Public top-level groups cannot be hosted on other Cells, only on the Legacy Cell.
- Several features available on GitLab.com are not supported on Cells at this iteration.

We will migrate a few small GitLab internal top-level groups to Organizations on another Cell using direct transfer. There will be migrations of Organizations between Cells for Cells 1.0. The specific top-level groups to be migrated will be identified in due course.

All existing top-level groups on GitLab.com are part of the default Organization.
A list of all GitLab top-level groups can be found in this Google doc (internal link).

We will create a new temporary Organization (GitLab Inc.) in a Cell (not Legacy Cell).

We will use direct transfer
or congregate to migrate select top-level groups belonging to GitLab Inc. from the default Organization in the Legacy Cell to the GitLab Inc. Organization on the other Cell. If Org mover is ready in time, we will use it instead.

All traffic to the top-level groups being migrated will be blocked for the duration of the migration. After the migration is complete, traffic will be resumed and redirected to the other Cell where the groups are located. They will be read/write and fully operational.

Cells 1.5

At this Cells iteration the focus will be moving top-level groups into Organizations and then migrating those Organizations from the Legacy Cell to other Cells.

We will establish a process for moving top-level groups between Organizations on the same Cell. This functionality is net new and will need to be developed in time for Cells 1.5. It is also a pre-requisite for Org mover to move an Organization between Cells. The work is tracked by epic 11711.

The solution will need to re-write the org IDs when a top-level group is moved to a new Organization.

A new solution is preferable to using the existing DT, because DT is more complex than re-writing IDs.

The new solution would also ensure that new IDs are only generated where necessary when a top-level group is moved into a new Organization.

This improves the experience for users, as it minimizes the changes users have to make to their local repositories, bookmarks, etc. to match the new IDs and paths.

After the top-level groups for a customer are in their own Organization on Legacy Cell), the Organization will be put into maintenance mode to prevent further changes to the Organization's data while it's being migrated. Org mover then moves the entire Organization from the Legacy Cell to another Cell. After the Org is on the other Cell, traffic routing for the Organization is switched to the other Cell and maintenance mode for the Organization is disabled. The new Organization will not be fully operational on the Other Cell.

Limitations:

- Only top-level groups can be moved between Organizations.
- It will only be possible to move top-level groups between organizations in the same Cell - This is acceptable because our primary use case involves creating a new Organization for a customer in Cell 1 and moving their top-level groups into this Organization. Groups will not need to be moved across Cells at this iteration.
- Public top-level groups cannot be hosted on other Cells, only on the Legacy Cell.

Note, GitLab top-level groups cannot be consolidated under a single GitLab Organization on the Cell hosting the GitLab Inc Organization at this point because many GitLab top-level groups are public groups. Public groups are only supported on the Legacy GitLab Cell. This will need to wait until the Cells-2.0 iteration.

Cells 2.0

At this iteration it is anticipated that we will have a fleet of Cells. Load across these Cells will need to be rebalanced as Organizations grow requiring a mechanism for moving Organizations between Cells. Org mover will be evolved to move Organizations between Cells.

There is also a need to move top-level groups between Organizations that are on different Cells. The primary use case is to support mergers between two customer Organizations. We do not currently have solution for this. We will in due course evaluate whether org mover should be evolved to move top-level groups between Cells and other approaches.

Limitations:

- Only top-level groups can be moved between organizations.

At Cells 2.0, we will consolidate all GitLab top-level groups under the GitLab Organization by merging the top-levels groups that are part of the GitLab Inc Organization with the main GitLab Organization - the long term home for all GitLab top-level groups.

All remaining top-level groups belonging to GitLab on the Legacy Cell will be moved to the GitLab Organization. The entire GitLab Organization will be moved to the Cell hosting the GitLab Inc Organization using org mover. Following this, the two Organizations (GitLab and GitLab Inc) will be consolidated by moving the top-level groups in the temporary GitLab Inc Organization into the GitLab Organization. The GitLab Inc org will then be deleted.

SECTION: engineering/architecture/design-
documents/cells/mutual_authentication_between_cell_services.md

{{< engineering/design-document-header >}}

Pre-Reads

- Internal TLS

Summary

We require that all communication between Cell services is secure and both parties identities are verified.

Goals

- Ensure that all communication between Cell services is encrypted, uniquely identified, authenticated, and authorized on both sides.
- Leverage the existing PKI Interface that is used for Internal TLS.
- Provide a clear implementation path for both server and client services.
- Enable secure service-to-service authentication without introducing unnecessary complexity.
- Support Authorization based on mTLS certificates where appropriate.

Scope

This document focuses specifically on implementing mutual TLS (mTLS) for Authentication and Authorization between Cell and its services.

Out of Scope

- Implementation of a service mesh (e.g., Istio) for encrypting traffic via mTLS is out of scope for the following reasons:
 - We already leverage TLS inside a cell using Internal TLS, so extending this existing blueprint to support external services is more consistent with our architecture.
 - While service mesh provides a transparent way for application developers to implement mTLS with external services, this approach introduces significant security risks when used as your sole authorization mechanism without additional application-level controls (such as JWTs). If a vulnerability exists, attackers could exploit it to use the client as a proxy to send unauthorized requests to the mTLS server. The fundamental issue is that service meshes authorize communications based solely on service identity rather than validating the legitimacy of individual requests within those services, making it inadequate as a complete authorization solution in the absence of request-level validation mechanisms.
- While TLS is used to secure communications between CDNs/load balancers and their backends, as well as between internal services, this scope explicitly excludes:
 - Communication from external clients to GitLab services.
 - Communication between services inside a cell and those outside a cell that are not managed cells services

Implementation Principles

- Developer Experience: mTLS implementation should be transparent to developers with minimal code changes.
- Security: Only authorized services should be able to communicate with one another.
- High Availability: Certificate rotation must occur automatically without service interruption.
- Scalability: Solution must work for all Cell services.

Requirements

Requirement	Description
Priority	
Security	Only authorized services can communicate with one another
High availability	Certificates can be rotated automatically without service interruption
Cells support	Can be used for all cells services
Authorization support	Application Developers can use mTLS for authorization
Provision certificates within seconds	We can create a new certificate in seconds
Multiple Protocol support	Support HTTP/1.1, HTTP/2, gRPC
Internal Traffic	All traffic between client and server services must remain within the cloud provider's network
Gradual Adoption	First allow traffic without a certificate to be

accepted	med
Auditable	Can validate all services are using secure
authentication via mTLS	med
Cloud-managed	Can be integrated with cloud services
med	

Non-Goals

- mTLS should not be considered for managing user level authorization

Design and implementation details

mTLS Architecture

mermaid

sequenceDiagram

participant client as Client

participant server as Server

client->>server: Client connects to server

server->>client: Server presents TLS certificate

client->>client: Client verifies servers certificate

client->>server: Client presents TLS certificate

server->>server: Server validates TLS certificate

client->>server: Communicates over encrypted TLS connection

server->>client: Communicates over encrypted TLS connection

mTLS Implementation Flow

From the Internal TLS Blueprint, we obtain end-entity client certificates from GCP Secrets Manager for the GKE cluster.

Server/Producer Configuration

The external service host requires these changes:

- Deploy the service behind an Internal Load Balancer to ensure the service is not publicly accessible
- Configure mTLS Client Authentication on the Load Balancer
 - Upload the Private Root CA Certificate to the Trust Config to enforce authenticated access only
- Configure [Private Service Connect] with the Load Balancer as a backend
- Set up the permissions for the Cell's GCP project to connect to the [Private Service Connect] endpoint of the Server.

Client/Consumer Configuration

The service consuming the external API requires these changes:

- Connect to the [Private Service Connect] endpoint using the VPC where the client service is

deployed

- Mount the certificate/key pair in the client application
- Update client code to establish mTLS connections through the [Private Service Connect] endpoint

The diagram below illustrates the complete request flow between a Pod in a Cell and an external service, including the supporting infrastructure:

For detailed implementation examples and proof-of-concept documentation of this architecture, refer to: <https://gitlab.com/gitlab-org/gitlab/-/issues/468640>.

Authentication and Authorization with mTLS

Authentication

mTLS authentication in our Cell services architecture works through explicit certificate loading and connection setup rather than transparent proxying:

- Certificate Loading: Each service must explicitly load its client certificate and private key from the filesystem. This is done in trusted code paths, as shown in the mTLS POC client code.
- Connection Establishment: The service must explicitly add the TLS credentials to outgoing requests, as demonstrated in the request creation code.
- Certificate Validation: The GCP Loadbalancer validates the client's certificate against the trusted CA, ensuring only services with valid certificates can connect.

Example of loading and using TLS credentials in a Go client:

```
go
tlsCredentials, err := loadTLSCredentials()
if err != nil {
    log.Fatalf("Failed to load TLS credentials: %v", err)
}

// Create a connection with the TLS credentials
conn, err := grpc.Dial(serverAddr, grpc.WithTransportCredentials(tlsCredentials))

...
// loadTLSCredentials loads TLS credentials from file paths provided in environment variables
func loadTLSCredentials() (credentials.TransportCredentials, error) {
    // Get paths to TLS files from environment variables
    serverCACertPath := os.Getenv("SERVERCACERT")
    if serverCACertPath == "" {
        return nil, fmt.Errorf("SERVERCACERT environment variable is not set")
    }

    clientCertPath := os.Getenv("MTLSCERTCHAIN")
    if clientCertPath == "" {
        return nil, fmt.Errorf("MTLSCERTCHAIN environment variable is not set")
    }
}
```

```

}

clientKeyPath := os.Getenv("MTLSKEY")
if clientKeyPath == "" {
    return nil, fmt.Errorf("MTLSKEY environment variable is not set")
}

// Load server CA certificate
serverCACertBytes, err := ioutil.ReadFile(serverCACertPath)
if err != nil {
    return nil, fmt.Errorf("failed to read server CA certificate file: %v", err)
}

certPool := x509.NewCertPool()
if !certPool.AppendCertsFromPEM(serverCACertBytes) {
    return nil, fmt.Errorf("failed to add server CA's certificate to pool")
}

// Load client certificate and key
clientCert, err := tls.LoadX509KeyPair(clientCertPath, clientKeyPath)
if err != nil {
    return nil, fmt.Errorf("failed to load client certificate and key: %v", err)
}

// Create the credentials and return it
config := &tls.Config{
    Certificates: []tls.Certificate{clientCert},
    RootCAs:     certPool,
}

return credentials.NewTLS(config), nil
}

```

Source: mTLS POC client code

Authorization

Authorization in our mTLS implementation occurs after successful authentication and relies on client identity information:

- Certificate-Based Identity: After authenticating the client connection, the server extracts identity information from the client's certificate for authorization decisions.
- GCP LoadBalancer Headers: We leverage custom mTLS headers passed by GCP LoadBalancer to the backend service, which contain pre-extracted certificate information.
- Header Processing: The server extracts these headers from incoming requests to determine the client's identity and permissions without needing to re-parse the certificate.
- Access Control Enforcement: Based on the extracted identity (typically the Common Name), the server determines whether the client is authorized to access the requested resource.

Example of extracting and using certificate information for authorization in a gRPC server:

```
go
md, ok := metadata.FromIncomingContext(ctx)
if !ok {
    return nil, status.Error(codes.Internal, "failed to get metadata")
}

// Get the value of X-Client-Cert-Subject-Dn
// Header keys in gRPC metadata are lowercase
clientCertDNs := md.Get("x-client-cert-subject-dn")

var cellName string
if len(clientCertDNs) > 0 {
    // Call the function to extract common name
    commonName, err := extractCommonNameFromSubjectDN(clientCertDNs[0])
}
```

Function to extract the Common Name from the Subject DN:

```
go
func extractCommonNameFromSubjectDN(base64SubjectDN string) (string, error) {
    // Decode base64
    derBytes, err := base64.StdEncoding.DecodeString(base64SubjectDN)
    if err != nil {
        return "", fmt.Errorf("failed to decode base64: %w", err)
    }

    // Parse the DER-encoded subject DN
    var rdnSequence pkix.RDNSequence
    , err = asn1.Unmarshal(derBytes, &rdnSequence)
    if err != nil {
        return "", fmt.Errorf("failed to parse ASN.1 DER encoding: %w", err)
    }

    // Convert to a Name
    var subject pkix.Name
    subject.FillFromRDNSequence(&rdnSequence)

    // Return the common name
    return subject.CommonName, nil
}
```

Source: mTLS Server Code

Advantages of mTLS over Alternative Authentication Methods

We chose mTLS as our primary mechanism for both authentication and authorization in our Cell services architecture for several key reasons:

1. Infrastructure-Managed Identity: mTLS allows us to leverage infrastructure-provided identity rather than application-managed tokens. This shifts the responsibility of identity management from application code to our infrastructure components, which are more specialized for this purpose.

1. Simplified Secret Management: Unlike token-based approaches (such as JWT tokens) that often require storing tokens as environment variables or in configuration files, mTLS certificates can be automatically provisioned, rotated, and managed by our existing PKI infrastructure. This provides several benefits:

- No hardcoded secrets in application code or environment variables
- Reduced risk of token leakage through logs or configuration dumps
- Standard certificate lifecycle management rather than custom token management

1. Multi-Service Compatibility: The mTLS approach scales effectively across multiple services without requiring service-specific implementation details. Each service follows the same pattern for authentication and authorization, providing a consistent security model across our entire Cell architecture.

1. Automated Certificate Rotation: Certificates can be rotated automatically without service interruption, which is often more complex with token-based approaches. Our existing certificate management infrastructure handles rotation seamlessly, reducing operational overhead.

1. Dual-Purpose Security: mTLS provides both encryption and authentication in a single mechanism, simplifying our security architecture compared to approaches that separate these concerns.

Security Considerations

- If an attacker gains Remote Code Execution (RCE) on a pod, they could access certificates and keys stored on the filesystem. This is an inherent limitation, as RCE generally compromises all security boundaries within the pod.
- The design focuses on preventing unauthorized service-to-service communication in scenarios where an attacker has limited access to manipulate network requests but not full system access.
- Certificate rotation and proper secret management help mitigate risks associated with potential certificate compromise.

Private Root CA Regional Resilience

Regional Limitations of Certificate Authority Service

Google Cloud Certificate Authority Service is a regional service, meaning that CA Service resources are stored in specific geographical regions and cannot be moved or exported after creation. As documented in Google Cloud's disaster recovery guidance, regional resources cannot withstand a regional failure, making them vulnerable to regional outages.

This regional limitation presents a specific risk for our Cell architecture:

- New Cell Provisioning Disruption: Having a single Private Root CA means that during a regional outage, when we want to create a Cell in a different region, we won't be able to as the Private Root CA would be unavailable, making Cell provisioning fail.

- Existing Cell Operations: Importantly, existing Cells would continue to operate normally during a regional CA outage, as they rely on already-issued certificates that remain valid and functional.

Multi-Regional Private Root CA Strategy

To achieve regional resilience and ensure continuous certificate operations for new Cell provisioning, we implement a multi-regional Private Root CA strategy where each Cell uses the Private Root CA located in its deployment region.

Current Implementation

We currently provision Private Root CAs in the following regions:

- us-east1 - For Cells deployed in the US East region
- us-central1 - For Cells deployed in the US Central region

Planned Expansion

The strategy includes expanding Private Root CA deployment to cover all primary regions where Cells will be deployed:

- Americas: us-east1, us-central1, us-west1, northamerica-northeast1
- Europe: europe-west1, europe-west3, europe-north1
- Asia Pacific: asia-northeast1, asia-southeast1, australia-southeast1

Implementation Architecture and Considerations

Each Cell is provisioned with certificates from the Private Root CA located in the same region:

- Regional Alignment: Cells deployed in us-east1 use the Private Root CA in us-east1
- Certificate Trust Configuration: All Cells, and the Server (Topology Service) must be configured to trust certificates issued by any of the regional Root CAs to maintain interoperability between Cells across regions
- Monitoring and Alerting: Comprehensive monitoring ensures rapid detection of regional CA failures and provides visibility into provisioning capabilities across regions
- Certificate Lifecycle Coordination: Ensure certificate rotation and lifecycle management works consistently across all regional CAs
- Regional Capacity Planning: Monitor certificate issuance capacity and usage across regions to ensure adequate resources for Cell provisioning demands

Benefits of Multi-Regional Approach

- New Cell Provisioning Resilience: Even if one region experiences an outage, new Cells can continue to be provisioned in other regions using their respective regional Private Root CAs
- Reduced Latency: Each Cell uses a geographically close Root CA for optimal performance during certificate operations
- Regional Independence: Each region operates independently, so a regional outage only affects new Cell provisioning in that specific region
- Regional Isolation: A regional outage is contained to that region and doesn't affect Cell

operations or provisioning in other regions

- Existing Operations Continuity: Existing Cells continue operating normally during regional outages, as they don't require new certificate issuance for ongoing operations

This multi-regional strategy ensures that our Cell provisioning capabilities remain resilient to regional outages while maintaining the security posture and operational efficiency of our Cell architecture. While existing Cells continue to operate during regional CA outages, this approach ensures we can continue expanding our infrastructure in unaffected regions.

Certificate Lifecycle Management

Certificate TTLs

Certificate Time-To-Live (TTL) values are configured according to industry best practices and our internal security requirements. The following TTL values have been established for our PKI infrastructure (discussed in detail here):

Certificate Type	TTL
-----	---
Root CA	10 years
Intermediate/Subordinate CA	5 years
End-entity certificates	13 months

These TTL values strike a balance between security requirements (limiting exposure time of compromised certificates) and operational overhead (frequency of rotation).

Certificate Rotation

To ensure continuous operation without interruption, certificates are proactively rotated before expiration:

- Intermediate CA certificates: Rotated 90 days before expiry, aligned with the Internal TLS blueprint.
- End-entity certificates: Rotated 60 days before expiry.

The rotation of intermediate and end-entity certificates is automated through the Instrumentor, which handles the provisioning and rotation of these certificates as implemented in:

- End-entity certificate rotation
- Intermediate certificate rotation

Root CA Rotation

Root CA rotation requires more careful orchestration. The rotation is performed every 5 years (or as needed) following Google Cloud's CA rotation best practices outlined in the Certificate Authority Service documentation.

The Root CA rotation process is summarized as follows:

1. Identify the CA pool containing the existing Root CA that is due to expire.

2. Create a new CA in the same CA pool in STAGED state.
3. Change the state of the new CA to ENABLED, enabling certificate issuance from both old and new CAs.
4. Update the Trust Store configuration on the Server (Topology Service) to accept requests authenticated with certificates issued by the new CA.
5. Update the Instrumentor to use the new Root CA for provisioning new Intermediate CA certificates.
6. Change the state of the old CA to DISABLED, preventing new certificate issuance while maintaining trust.
7. Wait until all clients have stopped using certificates issued from the old CA (either by waiting for maximum certificate lifetime or by monitoring client certificate usage).
8. Delete the old CA once all intermediate certificates have been issued using the new CA.

This carefully orchestrated process ensures zero downtime during Root CA rotation while maintaining the security integrity of the PKI infrastructure.

DNS Resolution for mTLS Server Communication

For mTLS to function correctly, clients must reach the server using the DNS name present in the server certificate's Subject Alternative Name (SAN) field. In our Cell architecture with [Private Service Connect], this presents a unique challenge as each Cell may have a different IP address for the same service.

Implementation Details

To ensure consistent DNS resolution across all Cells while maintaining proper certificate validation, we will implement the following approach:

```

mermaid
sequenceDiagram
    participant Client as Client in Cell
    participant KubeDNS as KubeDNS
    participant CloudDNS as CloudDNS Private Zone
    participant PSCIP as Cell-specific PSC IP
    participant ILB as Internal Load Balancer
    participant Service as Server Service
  
```

Note over PSCIP: PSC = Private Service Connect

```

Client->>KubeDNS: DNS lookup request
KubeDNS->>CloudDNS: Forward DNS query
CloudDNS->>KubeDNS: Return Cell-specific PSC IP
KubeDNS->>Client: Return PSC IP address
Client->>PSCIP: Connect to PSC IP
PSCIP->>ILB: Route to internal load balancer
ILB->>Service: Route to appropriate service instance
Service-->>ILB: Response
ILB-->>PSCIP: Response
PSCIP-->>Client: Response
  
```


1. Private CloudDNS Zone per Cell:

- Each Cell project will have its own CloudDNS Private Zone
- This zone will contain the same DNS name (e.g., topology-service.gitlab.net) for all Cells
- Each zone will resolve to the Cell-specific Private Service Connect IP that is dynamically reserved from the Cell's VPC when the PSC endpoint is created

2. DNS Resolution Flow:

- Client services use the standard DNS name in their requests
- KubeDNS forwards the request to CloudDNS Private Zone
- CloudDNS resolves the name to the Cell's specific [Private Service Connect] endpoint IP
- The request reaches the correct service through the [Private Service Connect] endpoint

3. Certificate Validation:

- The server certificate's SAN includes the standard DNS name
- Clients validate the certificate against this name, ensuring proper mTLS authentication

Advantages

- Consistent Naming: All Cells use the same DNS name to access services, simplifying configuration
- Certificate Compatibility: The DNS name matches the certificate's SAN, enabling proper mTLS validation
- Isolation: Each Cell maintains its own DNS resolution to its specific [Private Service Connect] endpoint
- Proven Solution: This approach is already implemented and tested in our Production environment for Vault services
- Infrastructure as Code: All DNS configurations are managed through Terraform

Implementation Reference

This implementation leverages our existing infrastructure patterns:

1. Service Exposure: The internal Load Balancer is exposed through [Private Service Connect] via serviceAttachment
2. Access Control: Projects are dynamically configured to connect to the service
3. DNS Configuration: Private CloudDNS zones are created in each consumer project

This approach works seamlessly with KubeDNS as the DNS provider for the cluster without requiring additional permissions or switching to CloudDNS as the cluster's DNS provider. This solution is already implemented and running in our Production environment for Vault services, with the following reference configurations:

- Service Exposure via Private Service Connect serviceAttachment
- Project Authorization for service access
- DNS Zone Configuration in consumer projects

[Private Service Connect]: <https://cloud.google.com/vpc/docs/private-service-connect/>

Supported clients & servers

Client	Server
GitLab	Topology Service

SECTION: engineering/architecture/design-documents/cells/proposal-admin_area_setting_synchronization_in_cells.md

{{< engineering/design-document-header >}}

Goals

We desire an automated solution for synchronizing application settings across multiple cells. This solution must integrate with the existing ring-based deployment model while supporting both global consistency and local customizations.

Currently, GitLab.com's transition to a distributed deployment model with cells organized in rings presents operational challenges. While Ring 0 (legacy monolith) uses Helm charts for settings control, Ring 1+ cells require manual SRE updates, creating operational overhead, configuration drift risk, and scalability concerns as the number of cells and rings expands.

Key Terminology

- Desired State: The intended configuration defined in version-controlled repositories (imperative configuration-as-code)
- Actual State: The current runtime configuration present in live cell instances
- Configuration Drift: When actual state diverges from desired state

Requirements

Requirement	Description
Configuration as Code	Store all settings in version-controlled repositories
Live states	Read the states from the current live cells
Ring-Based Integration	Build upon existing ring deployment model
Hierarchical Settings	Support inheritance from base · ring · cell overrides
Feedback Mechanism	Provide clear success/failure feedback for operations
Scalability Design	Handle increasing numbers of cells and rings efficiently
Secure Authentication	Authentication between solution and cell should have minimum privileges and can be rotated.
Minimal Secret Management	Reduce need to transmit or store secrets

Proposed Solution

To achieve this, a solution that leverages efficient concurrent execution and API-based communication with cells would be beneficial. While several implementation approaches might be viable (discussed later), extending the existing ringctl tool represents one potential path.

Implementation Ideas

To address the requirements outlined above, a potential settings synchronization system could include:

API Considerations

The Backend team would be essential partners in developing secure API endpoints for settings management. Key requirements for this API:

- Internal Tool Access Only: API accessible only by the synchronization tool (ringctl or similar), not by customers
- Scoped Authentication: Authentication must be scoped specifically to cell settings operations
- Rotatable Credentials: Authentication mechanism must support credential rotation
- Future Design Flexibility: Specific HTTP methods (GET/PUT/PATCH) and detailed endpoint design can be determined during implementation phase

Authentication Strategy

JWT Secret Management: JWT signing secrets would be stored in Vault and made available to the Tissue project's CI environment during ring deployment operations. This approach leverages existing secret management infrastructure while maintaining security boundaries between deployment tooling and target cells.

Environment-Based Secrets: JWT signing secrets would be distinguished by environment (non-prod/prod) rather than per-ring or per-cell. This approach balances security isolation with operational simplicity, allowing for environment-level credential rotation while avoiding unnecessary key management complexity.

Unified Authentication Approach: The JWT-based authentication strategy could serve as a foundation for other cross-cell operations including feature flag controls and rate limit management. This creates a consistent authentication pattern for internal tooling while avoiding the proliferation of different authentication mechanisms across operational tools.

This approach prioritizes security and operational requirements while allowing implementation details to be refined collaboratively with the Backend team.

Settings Structure

A hierarchical structure that supports inheritance could be organized as follows:

```
bash
settings/
  allcells.yml      Base settings for all cells
  rings/
    ring1.yml       Ring-specific overrides
    ring2.yml
  cells/
    cella.yml       Cell-specific overrides
    cellb.yml
```

Potential Implementation Approach

One approach to consider would be extending the ringctl tool with settings synchronization capabilities:

bash

ringctl setting-sync --ring=<ringname> [options]

This implementation could potentially:

1. Support concurrent operations across multiple cells in a ring
2. Implement a get-then-update pattern that reads actual state from cells and compares against desired state from version control
3. Provide detailed feedback on operations
4. Include appropriate error handling and resilience mechanisms

Possible Architecture

mermaid

flowchart TB

TR[Tissue Repository] --> |Git-based workflows| RC[ringctl setting-sync --ring=N]

RC --> |Parallel execution against| Caaaxyz

RC --> |Parallel execution against| Czzzbbb

subgraph "Ring N"

direction LR

Caaaxyz["Cell aaaxyz"]

Czzzbbb["Cell zzzbbb"]

end

subgraph "Settings Inheritance"

direction TB

B[Base Settings] --> |Override| RS[Ring Settings]

RS --> |Override| CS[Cell Settings]

end

Caaaxyz --> API1

Czzzbbb --> API2

subgraph "API Layer"

direction TB

API1[Settings API] --> |Idempotent| APP1[Application]

end

subgraph "API Layer"

direction TB

API2[Settings API] --> |Idempotent| APP2[Application]

end

Potential Benefits

- Reduces Manual Configuration: Lightens operational overhead and reduces potential for human error
- Ensures Consistency: Contributes to preventing configuration drift between cells
- Builds on Existing Workflows: Leverages experience with the ring-based deployment model
- Enhances Flexibility: Enables both global consistency and cell-specific customizations when needed
- Supports Scalability: Could handle increasing numbers of cells and rings through concurrent execution
- Provides Atomicity: Helps ensure all cells in a ring are updated together
- Maintains Configuration Integrity: Compares actual state from live cells against desired state in version control to detect and correct drift
- Strengthens Security: Uses service account authentication instead of personal tokens
- Provides Visibility: Offers clear success/failure information for each operation

Considerations and Challenges

- API Development Collaboration: Partnership with the Backend team would be needed to develop appropriate internal API endpoints
- Complexity Management: New elements to tooling and deployment process would require thorough documentation
- Cross-Team Coordination: Collaboration between SRE, Backend, and Platform teams would be beneficial
- New Failure Patterns: Settings deployment might encounter new types of failures requiring handling strategies
- Service Account Security: Thoughtful handling of service account credentials would be necessary
- Performance Considerations: Concurrent settings updates might have performance implications worth exploring
- Validation Needs: Verification that settings are correctly applied would be important
- Consistency Management: Temporary inconsistencies during updates might require specific handling approaches

Alternative Approaches Worth Considering

Several options could address this challenge:

Terraform-based Configuration Management

Approach: Leverage Terraform for managing settings across cells.

Considerations:

- Benefits from being an established infrastructure-as-code tool
- Offers strong state management capabilities
- Provides a declarative approach to configuration
- Presents some challenges with state file locking in distributed environments
- Might introduce complexity for application-level configuration
- Has different strengths than what might be needed for dynamic application settings
- Could experience state drift from actual configurations

GitOps Approach (ArgoCD/Flux)

Approach: Implement GitOps tools to automatically apply configuration from Git repositories.

Considerations:

- Provides an imperative approach with Git as the desired state
- Includes built-in reconciliation mechanisms
- Creates a good audit trail via Git history
- Might have interactions with existing Instrumentor and ringctl patching to consider
- Would add new elements to the deployment architecture
- Could potentially support customization requirements with some adaptation

Custom Synchronization Service

Approach: Develop a tailored service dedicated to settings synchronization.

Considerations:

- Could be designed specifically for the requirements
- Might optimize for the exact use case
- Would require development resources
- Would create a new service to maintain and operate
- Could leverage existing functionality where appropriate

API-based Configuration Management

Approach: Build on existing tools to use API-based execution for settings synchronization.

Considerations:

- Builds on existing tools and knowledge
- Could support idempotent operations
- Avoids SSH dependencies through API-based execution
- Would enhance existing tools with new capabilities
- Would require collaboration on new API endpoints

Technical Areas to Explore

API Design Collaboration

When discussing the settings API with the Backend team, several areas merit exploration:

1. API Structure Options:

- Creating dedicated endpoints for settings management
- Extending existing APIs to support settings operations
- Considering appropriate URL structure and naming conventions

2. Authentication Approaches:

- Service account authentication options
- Appropriate permission scopes
- Security considerations for internal APIs

3. Request/Response Formats:

- Schema validation approaches
- Error handling and reporting
- Support for partial updates

Service Account Security

Secure credential management for service accounts requires consideration of:

1. Creating dedicated service accounts with appropriate permissions
2. Implementing secure secret storage and rotation
3. Establishing auditing and monitoring for service account usage

Resilience Strategies

The synchronization process would need to handle various scenarios:

1. Network interruptions during synchronization
2. Partial success scenarios where some cells update successfully while others don't
3. Rollback options for failed deployments
4. Detailed error reporting and logging

Cross-Cell Verification

Verification mechanisms to ensure settings are properly synchronized could include:

1. Status reporting for each cell after synchronization attempts
2. Periodic consistency checks across cells
3. Reconciliation processes for detected inconsistencies

Scaling Approaches

As the number of rings and cells grows, considerations include:

1. Resource usage during concurrent operations
2. Timeout and retry mechanisms for network issues
3. Batching strategies for very large deployments
4. Monitoring and alerting for synchronization events

Settings Inheritance

The hierarchical settings structure raises questions about:

1. How conflicts between base, ring, and cell settings might be resolved
2. Which settings could be overridden at which levels
3. How to handle removal of settings (null values vs. explicit removal)
4. Documentation of inheritance patterns for operators

Implementation Pathways

Based on exploration of the problem space, key considerations for moving forward include:

1. Backend API Collaboration:
 - Working with the Backend team on secure API endpoints
 - Implementing appropriate authentication for service accounts
 - Supporting retrieval and updating of settings
2. Settings Structure:
 - Storing settings in Git repository with hierarchical organization
 - Implementing inheritance from base · ring · cell
 - Supporting variable substitution for environment-specific values
3. Synchronization Strategy:
 - Building on existing tools with settings synchronization capabilities
 - Using concurrent execution for efficiency across multiple cells

- Implementing get-then-update pattern to minimize unnecessary changes
- Providing detailed reporting on operations

4. Implementation Options:

- Several approaches could work effectively, including:
 - Direct API calls from a synchronization tool
 - Configuration management tools that operate via API rather than SSH
 - Synchronization logic that builds on existing tooling

Each approach has its merits. The final implementation decision should consider available expertise, integration with existing systems, and operational requirements.

SECTION: engineering/architecture/design-documents/cells/routable_tokens.md

This document describes design goals and architecture of Routable Tokens used by Cells. This document explicitly focuses on objectives of Phase 4.

Purpose

GitLab uses machine-generated tokens extensively to provide various ways for Users/Services to interact with GitLab, for example, the REST API Authentication and the Token Overview. Tokens have different scopes as for example User, project, and group

HTTP Routing Service require the tokens to be routable, so it can route the request to the correct Cell.

Goals

This documents tries to describe the following goals:

- Routable tokens to be decoded by HTTP Router.
- Capture and describe usage of existing tokens.

Non-goals

This document is not meant to re-assess the need for the following non-goals:

- The need for the secret in the first place, and their lifecycle or how they are managed by the application.
- Define support for many encryption keys
- Define the pattern allowing online encryption keys rotation
- Document how to deprecate and remove legacy encryption keys, or legacy strategies
- Unify storing of secrets and tokens
- Move away from attrencrypted to use a single secrets framework
- Introduce transit/shared key to be used with Org Mover

Those non-goals are meant to be solved with a new design document describing secrets management

by the application.

Tokens and cookies

The list of tokens at a current moment is shared as a Google Spreadsheet to ease editing.

Secrets

This document focuses only on Routable Tokens, not secrets stored at rest, like CI Variables.

Existing implementation

TokenAuthenticatable

Majority of the tokens in application use TokenAuthenticatable framework making it easy to change how the token is generated. Only some of the tokens use a custom tokengenerator, as in the following example:

```
ruby
module Clusters
  class AgentToken < ApplicationRecord
    TOKENPREFIX = "glagent-"

    add_authentication_token_field :token,
      encrypted: :required,
      tokengenerator: -> { Devise.friendly_token(50) },
      format_with_prefix: :glagentprefix

    def glagentprefix
      TOKENPREFIX
    end
  end
end
```

Proposal

This proposal is to make all tokens to encode routable information about object to which the token is attached. This document does focus specifically first on tokens that are required to be made routable in the Phase 4: Personal Access Token, CI/CD Job Token and Runner Authentication Token:

Currently tokens are generated with the following pattern: <prefix><random-string>. The Routable Token would change this to <prefix><base64-payload>.<token-version>.<base64-payload-length><crc32>.

Specification

- The Routable Token would change the <random-string> to become <base64-payload>.<token-version>.<base64-payload-length><crc32>.

- The <base64-payload> is a base64-encoded string composed of 3 parts: <random-bytes><routing-payload><routing-payload-length>.
- The <random-bytes> is a set of random bytes to ensure a high entropy, so the token cannot be forged.
- The <routing-payload> is a line-delimited string in the form of
c:3w5e11264sgsfng:3w5e11264sgsfnp:3w5e11264sgsf.
- It contains information that will allow the HTTP Router to route requests to the cell where the token is intended to be used.
- Each routing line starts with a character indicating a type of value it describes. The type and value are separated by the : character.
 - Integer values must be encoded as base36 string for space efficiency.
 - Lines are sorted alphabetically (e.g. c: comes before g: etc.).
- The <routing-payload-length> is 1 byte (8-bit unsigned (unsigned char)) (i.e. <integer>.pack("C")) that stores the length of <routing-payload>.
- The <token-version> is an integer represented in base36, using two alphanumeric characters (e.g., 00, 0a, zz), generated using <integer>.tos(36).rjust(2, '0'). This format supports versioning from 0 to 1295.
- The <base64-payload-length> is an integer represented in base36, which we use 2 bytes and pad with 0 on the significant digit (i.e. <integer>.tos(36).rjust(2, '0')) that store the length of <base64-payload>.
- The <crc32> is an integer represented in base36, which we use 7 bytes and pad with 0 on the significant digits (i.e. <integer>.tos(36).rjust(7, '0') in Ruby) that store a CRC32 checksum of <prefix><base64-payload>.<base64-payload-length>.

Constraints

- Minimum number of routing parts is 1.
 - An exception should be raised if no routing part is defined.
- Maximum number of routing parts is 10.
 - An exception should be raised if more than 10 routing parts are defined.
- Minimum size of <routing-payload> is 3 bytes (i.e. o:1).
 - An exception should be raised if <routing-payload> is smaller than 3 bytes.
- Maximum size of <routing-payload> is 159 bytes: 'c:3w5e11264sgsf'.size 10 + (10 - 1) (see Maximum token length).
 - An exception should be raised if <routing-payload> is bigger than 159 bytes.
- Valid routing part keys are currently c, g, o, p, u, t. Any other keys should raise an exception.
- Minimum number of random bytes is 16.
 - This is arbitrary to ensure a high entropy.
- Maximum number of random bytes is 65: (maximum bytes before encoding) - (max size of <routing-payload>) - (size of <random-bytes-length>) = 225 - 159 - 1 = 65
 - This ensures we can always encode the biggest <routing-payload>.
- Minimum size of <base64-payload> is 27 bytes (20 bytes before encoding: (min size of <random-bytes>) + (min size of <routing-payload>) + (size of <routing-payload-length>) = 16 + 3 + 1 = 20)
 - Maximum size of <base64-payload> is 300 bytes (225 bytes before encoding).
 - This is arbitrary and should be enough to carry all the information we need for now.
 - An exception should be raised if <base64-payload> is bigger than 300 bytes.
- Minimum size of prefix is 0 bytes.
- Maximum size of prefix is 20 bytes.

- An exception should be raised if prefix is bigger than 20 bytes.
- Minimum size of token is 40 bytes: (min size of <base64-payload>) + (size of '.') + (size of <token-version>) + (size of '.') + (size of <base64-payload-length>) + (size of <crc32>) = 27 + 1 + 2 + 1 + 2 + 7 = 40
- Maximum size of token without prefix is 313 bytes: (max size of <base64-payload>) + (size of '.') + (size of <token-version>) + (size of '.') + (size of <base64-payload-length>) + (size of <crc32>) = 300 + 1 + 2 + 1 + 2 + 7 = 313
- Maximum size of token with prefix is 333 bytes: (max size of prefix) + (max size of <base64-payload>) + (size of '.') + (size of <token-version>) + (size of '.') + (size of <base64-payload-length>) + (size of <crc32>) = 20 + 300 + 1 + 2 + 1 + 2 + 7 = 333

Additional information

- The generated token is stored in whole as-is, and is validated against its full value by the Rails application. This doesn't change from the current logic.
Note that this is contrary to JWT which usually a signature is used to validate authenticity of the token itself.
- The Rails application should never decode <base64-payload> and only use the whole token as-is for authentication purposes.
- The tokens as stored and validated by the application would not change.
- The ability to decode <base64-payload> is a feature reserved for the HTTP Router.
- The <routing-payload> can be easily retrieved from <base64-payload> (pseudo-code):
 1. Retrieve <base64-payload-length> and convert it from base36 to integer (i.e. <token>[-9, 2].toi(36) in Ruby. parseInt(<token>.slice(-9, -7), 36) in JavaScript)
 1. Retrieve <base64-payload> (i.e. <token>[(-10 - <base64-payload-length>), <base64-payload-length>] in Ruby. <token>.slice(-10 - <base64-payload-length>, -11) in JavaScript)
 1. Base64-decode <base64-payload> to get <payload> (i.e. Base64.urlsafe_decode64(<base64-payload>) in Ruby. atob(<base64-payload>.replace(/g, '/').replace(/-/g, '+')) in JavaScript)
 1. Retrieve <random-bytes-length> and unpack it to a 1 byte unsigned char (i.e. <payload>[-1].unpack1("C") in Ruby. payload.slice(-1).charCodeAt(0) in JavaScript)
 1. Retrieve <routing-payload> (i.e. <payload>[...-<random-bytes-length> - 1] in Ruby. payload.slice(0, -<random-bytes-length> - 1) in JavaScript)
- Secret detection tools will be able to check authenticity of tokens offline, and without base64-decoding the token (i.e. Zlib.crc32(<token>[...-7]) == <token>[-7, 7].toi(36) in Ruby), since the checksum is included as the last 7 characters of the token.
- The TokenAuthenticatable framework will be updated to allow generating routable tokens.
- Secret detection tools will need to be changed to accommodate the longer and variable token length, and the new .<base64-payload-length><crc32> suffix:
 - app/assets/javascripts/lib/utils/secretDetectionPatterns.js
 - The GitLab Secret Detection gem
 - GitLab secrets SAST analyzer
 - Tokinator

Implementation

Current implementation can be found at <<https://gitlab.com/gitlab-org/gitlab/-/blob/master/lib/authn/tokenfield/generator/routabletoken.rb>>.

Minimum token length

Here's an example of a token having minimum id for a single routable part, with no prefix showing the minimum length of a token (40 bytes):

```
text
EL25dAKXDUtqLnIQHEpkG86MQM.01.0r1ajj2i3
```

Here is its routing payload:

```
text
o:1
```

Maximum token length

Here's an example of a token having maximum ids for all possible routable parts, prefixed with the longest prefix (20 bytes) showing the maximum length of a token (333 bytes):

```
text
+++++LB4hkKqprRab1Y3A72WqTPSDo2FS1t1qgJTIhL2Okp90C8s8cL55xHQtmr-eVFu
qS3RI2ku4fBjOjN3NWUxMTI2NHNnc2YKZzozdzVIMTEyNjRzZ3NmCmg6M3c1ZTExmJY0c2dzZgpqOjN3N
KazozdzVIMTEyNjRzZ3NmCmw6M3c1ZTExmJY0c2dzZgptOjN3NWUxMTI2NHNnc2YKbzozdzVIMTEyNjRz
ExmJY0c2dzZgp1OjN3NWUxMTI2NHNnc2af.01.8c0kai99b
```

Here is its routing payload:

```
text
c:3w5e11264sgsf
g:3w5e11264sgsf
h:3w5e11264sgsf
j:3w5e11264sgsf
k:3w5e11264sgsf
l:3w5e11264sgsf
m:3w5e11264sgsf
o:3w5e11264sgsf
p:3w5e11264sgsf
u:3w5e11264sgsf
```

Note that 3w5e11264sgsf is (264-1).tos(36) which is the largest number for a bigint. Also note that l, k, j, h, m are not actual routing keys and are only there to demonstrate the maximum theoretical size of a token.

In practice, it's unlikely that all routable parts would be set, but this can be useful to know that maximum token length (e.g. for secret detection scripts).

Meaning of fields

Since the payload holds a structured information, each single letter has a particular meaning. The following fields are always required:

- c: Cell ID
- o: Organization ID

The following fields are optional. Each specific tokens can include them if needed:

- g: Group ID
- p: Project ID
- u: User ID
- t: Runner type (e.g., t:1 for instance type, t:2 for group type, and t:3 for project type)

It's recommended that for tracing and observing purpose, we can include the most important information for the specific token. For example, for a user token, we also include user id. For a project token, we also include the project id.

Adding Classify to Topology Service

It is strongly desired that Classify endpoint of Topology Service could accept routable token payload in full (except for r), and make the best routing decision on its own based on available information.

The Topology Service once it receives the full payload should look at making routing decision based on ID cardinality, preferring to search by Project ID, Group ID, Organization ID, User ID, Cell ID. As such adding support for new identifiers in Topology Service will not change HTTP Router implementation, and will make those be automatically supported by Topology Service.

The request sent to Topology Service should never include r field that is meant to ensure that token is truly unique. Sending r is not required, and by us not sending it reduces attack surface on Topology Service, since Topology Service without r cannot reconstruct the token.

```
proto
enum ClassifyType {
  ROUTABLETOKEN = 3;
}

message ClassifyRequest {
  ClassifyType type = 2;
  oneof value {
    string str = 3;
    map<string,string> routabletoken = 4;
  };
}

service ClassifyService {
  rpc Classify(ClassifyRequest) returns (ClassifyResponse) {
    option (google.api.http) = {
```

```

    post: "/v1/classify",
    body: ""
  };
}
}

```

Assuming that we send to classify based on payload from the previous point we would send the following request to Topology Service:

```

ruby
classifyservice.classify(
  ClassifyRequest.new(
    type: ClassifyType.ROUTABLETOKEN,
    routabletoken: {
      "c": "100",
      "o": "1",
      "u": "100"
    }
  )
)

```

The Topology Service routes by the available information following this precedence:

1. o
1. g
1. p
1. u
1. c

Integration into Token Authenticatable

The Routable Token is meant to be integrated into TokenAuthenticatable as a first-class supported syntax. Once the routabletoken: is used, it will change how the payload is generated. The usage of routabletoken: should not be used with generator: which is sometimes used to generate a random string of a different format.

```

ruby
class PersonalAccessToken
  addauthenticationtokenfield :token,
    encrypted: :required,
    formatwithprefix: :prefixfromapplicationcurrentsettings,
    routabletoken:
      if: ->(tokenownerrecord) { Feature.enabled?(:routabletoken, tokenownerrecord.user) },
      payload: {
        o: ->(tokenownerrecord) { tokenownerrecord.organizationid.tos(36) },
        u: ->(tokenownerrecord) { tokenownerrecord.userid.tos(36) }
      }
end

```

Integration into Rules engine of HTTP Router

We intentionally encode more information to be able to change over time the routing criteria by modifying HTTP Router rules. The HTTP Router would introduce a stage of transforming information: transform.

```
json
[
  {
    "match": [
      {
        "type": "header",
        "name": "private-token",
        "regexvalue": "^glpat-(?<payload>.)$"
      }
    ],

```